TunnelForge: Design and Experimental Evaluation of a Go-Based Reverse HTTP Tunnel over WebSocket

Soham Agarwal
Roll No. 202452332

Prem Sharma
Roll No. 202452328

Aghnivesh Thottumkara
Roll No. 202451175

Department of Information Technology
Indian Institute of Information Technology
Under the supervision of Dr. Rishabh Gupta

Abstract- Demonstrating a local web application to an external caller requires a reachable endpoint and a mechanism for forwarding requests across network boundaries. This paper presents TunnelForge, a Go-based reverse HTTP tunnel that carries structured request and response messages over an outbound WebSocket connection. The implementation combines temporary tunnel registration, token authentication, request correlation, bounded concurrency, cancellation, and connection recovery within a single-relay architecture. We evaluate functional behavior and compare direct HTTP with tunnelled HTTP using a reproducible, closed-loop experiment. Across 9,216 measured requests with 1 KiB and 64 KiB payloads, all responses matched the submitted bytes and no request failed. For one outstanding request, median tunnel latency was 0.182 ms and 3.016 ms, respectively, compared with 0.035 ms and 0.076 ms for direct HTTP. These loopback measurements characterize local implementation overhead rather than Internet performance. The findings illustrate the trade-off between an inspectable JSON protocol and the serialization and buffering costs of a compact educational tunnel.

Index Terms- reverse HTTP tunnel, WebSocket, Go, request correlation, concurrency, developer tools, experimental evaluation.

I. INTRODUCTION

Local development environments commonly host web interfaces, REST APIs, and webhook receivers on loopback addresses. These services are convenient for development but do not automatically provide an endpoint that a remote collaborator or external webhook provider can reach. Inbound port forwarding, public deployment, and reverse tunnelling are possible ways to address this gap. A reverse tunnel changes the direction of connection establishment: an agent near the application connects outward to a reachable relay, which forwards incoming requests over the established connection.

WebSocket provides a persistent bidirectional channel after an HTTP opening handshake [1]. It is therefore suitable for carrying multiple application-level request and response messages over one client-initiated connection. However, selecting WebSocket does not solve HTTP forwarding by itself. The application must still represent HTTP data, match out-of-order responses, manage disconnected peers, avoid concurrent socket writes, and terminate abandoned work. HTTP semantics also require attention to status codes, header multiplicity, redirects, and intermediary behavior [2].

TunnelForge addresses these concerns in a small implementation intended for a college systems project. A command-line client connects to a relay, authenticates, and receives a temporary routing identifier. Public-side requests are represented as versioned JSON messages, delivered to the client, and reconstructed as requests to the local application. The response returns with the same request identifier. Here, “public-side” identifies the relay-facing interface; the evaluated deployment is local and does not demonstrate Internet reachability.

The paper investigates three questions. RQ1 asks whether the implementation preserves the tested HTTP request and response properties, including arbitrary body bytes. RQ2 asks whether concurrency and failure handling maintain correct request ownership and release abandoned work. RQ3 asks how the tunnel changes latency and throughput relative to direct HTTP under controlled local workloads. The contribution is an implementation study with reproducible evidence, not a claim that reverse tunnelling is a new networking technique.

II. BACKGROUND AND RELATED WORK

Managed tunnelling services already provide mature ways to expose local applications. The ngrok documentation describes sharing localhost services through public endpoints and includes webhook testing and traffic inspection use cases [3]. The open-source frp project offers a client/server reverse proxy with broader protocol support and configurable forwarding features [4]. These systems establish the practical relevance of the problem. They are not performance baselines in this study, because no controlled measurements of their deployments were collected.

TunnelForge deliberately exposes the forwarding mechanism in a smaller codebase: a single relay, an in-memory registry, a Go client, and explicit JSON message types. It does not attempt feature parity with managed gateways or multi-protocol proxies. The comparison is architectural: established tools emphasize operational breadth, while this project emphasizes inspectability, constrained HTTP behavior, and repeatable demonstrations of concurrency and failure recovery.

The protocol combines standardized building blocks. JSON supplies the structured interchange format [5]; Base64 supplies a textual representation for arbitrary octets [6]. Go's encoding/json package encodes byte slices as Base64 strings [7]. These choices make protocol examples easy to inspect, but introduce expansion and processing overhead that motivates RQ3. The study evaluates that trade-off without attributing all measured overhead to a single mechanism.

III. SYSTEM MODEL AND DESIGN

A. Components and Deployment Assumptions

The system has four logical roles: an HTTP caller, the relay, the tunnel client, and the local application. Figure 1 shows the data path and the two relay-side state structures. The client initiates the WebSocket connection, allowing the application host to avoid accepting a new inbound connection from the relay. A real remote deployment still requires the relay itself to be reachable and the intervening network to permit the outbound connection. NAT traversal is an architectural motivation, not an experimentally verified result of the loopback setup.

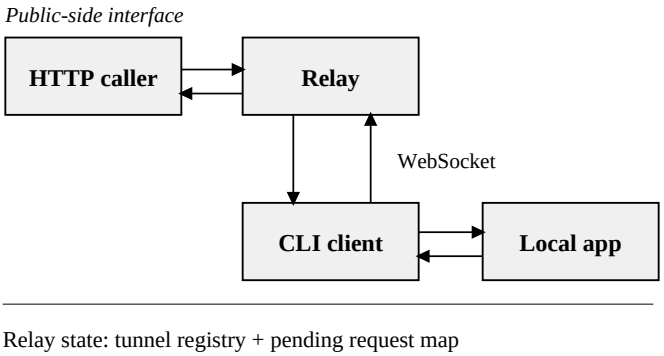

Fig. 1. Logical architecture. The client initiates the WebSocket connection; all experiment endpoints use loopback sockets.

The relay exposes /tunnels for registration and /health and /ready for process availability. In path-routing mode, it associates /t/<identifier>/... with the connected client and removes that prefix before forwarding. Subdomain routing is also implemented, but requires suitable DNS and TLS configuration outside the evaluated environment. The local application remains a normal HTTP server and does not import the tunnel protocol.

The registry maps a public routing identifier to a tunnel record and connection owner. Public routing identifiers use five random bytes rendered as hexadecimal; internal tunnel and request identifiers use sixteen random bytes. Registration retries identifier generation when a collision occurs. Random routing names reduce predictability but are not a substitute for access control. Disconnecting a client removes its registry entries and cancels its pending requests. No tunnel names or connection objects survive a relay restart.

B. Protocol Representation

Each message carries a version, a type, an optional request_id, and a typed payload. The decoder rejects unsupported versions, unknown message types, unknown JSON fields, missing required values, and invalid payload shapes. Table I summarizes the implemented messages. Registration and heartbeat messages are connection-scoped. HTTP request, response, and cancellation messages carry request identifiers so independent requests can coexist on one socket.

TABLE I APPLICATION PROTOCOL MESSAGES

Type	Scope and purpose
register	Connection: submit token
registered	Connection: return tunnel ID and URL
http_request	Request: method, path, query, headers, body
http_response	Request: status, headers, body
cancel	Request: cancel local HTTP context
heartbeat	Connection: application liveness message
error	Connection: registration/protocol failure

HTTP request payloads contain method, path, query, multi-valued headers, and body bytes. Responses contain status code, multi-valued headers, and body bytes. The decoder reconstructs native Go values before the local client performs an HTTP exchange. This is application-layer reconstruction, not byte-for-byte forwarding of the original HTTP wire representation. For example, the local request targets the configured host, while the original relay-facing host is provided as forwarding metadata.

C. Binary Data and Message Limits

Using byte slices avoids assuming that an HTTP body is valid UTF-8. Go's JSON encoder and decoder apply Base64 when transporting those slices [7]. For a non-empty body containing B bytes, the encoded body length is given by (1), before JSON fields and WebSocket framing are added:

$$L(B) = 4 \operatorname{ceil}(B / 3) \quad (1)$$

Thus, a 1,024-byte body becomes 1,368 Base64 characters, and a 65,536-byte body becomes 87,384 characters. This approximately one-third expansion applies in each direction when a request body is echoed as a response. It is a deterministic representation cost; measured latency also includes copying, parsing, scheduling, and socket I/O. The implementation caps each body at 1 MiB and each WebSocket message at 2 MiB. Header and envelope sizes consume part of the latter allowance.

D. Request Ownership Invariant

Before dispatch, the relay allocates a pending entry that binds a request ID to its tunnel ID, result channel, and cancellation function. A response is accepted only if both the request ID exists and its owner matches the responding tunnel. Resolving a request removes the entry and produces one terminal result. Timeouts, caller cancellation, and disconnects use the same cleanup concept. This prevents a different authenticated connection from completing another tunnel's request, even if it obtains the identifier. It does not provide tenant isolation beyond that specific ownership check.

IV. IMPLEMENTATION AND FAILURE HANDLING

A. End-to-End Forwarding Algorithm

The public handler first identifies the tunnel and connected client. It reads the request body within the configured bound, allocates a pending request, and enqueues an HTTP-request message. The client decodes that message and dispatches the local request. After reading the bounded response body, it returns the response payload with the original request ID. The relay validates ownership, restores the tested end-to-end headers, writes the status code, and returns the response bytes. Figure 2 summarizes the successful path and cancellation branch.

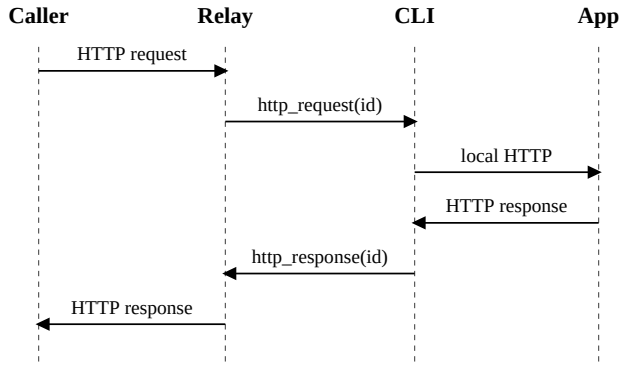

Fig. 2. Successful forwarding sequence. A timeout or caller disconnect instead triggers cancel(id) from relay to client and cancellation of the local HTTP context.

The implementation removes hop-by-hop headers and any additional header names listed in Connection. It retains header value lists so repeated Set-Cookie fields are not collapsed. Redirect following is disabled in the local HTTP client; otherwise an upstream 302 could be replaced by the result of a second request. Automatic decompression is also disabled to avoid silently changing the representation during forwarding. These are implementation decisions consistent with preserving the tested intermediary behavior [2], not a claim of complete HTTP conformance.

B. Concurrency and Resource Bounds

The client uses goroutines for concurrent local requests, with a semaphore that permits at most 64 active exchanges. When no slot is available, it returns a 503 response rather than waiting indefinitely for admission. A mutex serializes client writes. The relay uses one writer pump for each WebSocket connection and an outgoing queue with 128 entries. This follows the Gorilla WebSocket requirement for at most one concurrent application writer and one concurrent reader [8].

The relay limits its pending-request map to 1,000 entries. That bound and the client semaphore reduce unbounded request growth, but they do not establish a strict whole-process memory ceiling: bodies, JSON buffers, headers, queues, and transient copies consume additional memory. Table II lists the configured values. The experiment uses at most 32 closed-loop callers, so it exercises concurrent matching below the configured 64-request admission threshold rather than proving overload behavior at saturation.

TABLE II IMPLEMENTED RESOURCE AND TIMING LIMITS

Parameter	Value
HTTP body / WebSocket message	1 MiB / 2 MiB
Active local requests per connection	64
Relay pending requests / outgoing queue	1,000 / 128
Relay / local HTTP timeout	30 s / 35 s
Application heartbeat / relay ping	20 s / 25 s
Socket read deadline / backoff cap	60 s / 30 s

C. Cancellation, Liveness, and Recovery

Each pending request has a 30-second relay timeout. A timeout or caller disconnect removes the pending result and sends a cancellation message to the client. The client locates the corresponding context and cancels the local HTTP exchange. Cancellation is best-effort over the connection: it cannot undo a side effect already committed by the local application. The local transport has a 35-second fallback timeout, allowing the relay deadline to normally produce the intended 504 response first.

The relay emits WebSocket pings every 25 seconds and refreshes a 60-second read deadline on pong receipt. The client responds to pings and refreshes its own deadline. Separate application heartbeat messages are emitted every 20 seconds. After an unexpected disconnect, the client retries with exponential backoff capped at 30 seconds; successful registration resets the delay. Authentication rejection is terminal rather than retried. A reconnect creates a fresh temporary URL, so previously distributed URLs are not recovered.

D. Authentication and Security Boundary

The relay executable requires a shared token, compares the submitted value in constant time, and flushes an authentication error before closing a rejected connection. The embedded library can be instantiated without a token for controlled tests. Token registration authenticates tunnel creation; it does not authenticate every HTTP caller visiting the resulting URL. Therefore a publicly reachable tunnel can expose its application to anyone who obtains the address. HTTPS/WSS is required for a real deployment carrying secrets; this study used plaintext loopback connections and does not claim encrypted benchmark results.

V. EVALUATION METHODOLOGY

A. Experimental Environment and Artifact

Measurements were collected on 29 October 2026 on a single macOS 27.0 arm64 host using Go 1.26.5. The runtime reported 10 logical CPUs and GOMAXPROCS=10. The workload generator, sample HTTP application, relay, and tunnel client ran in one Go process with separate goroutines and actual loopback TCP sockets. The WebSocket dependency was Gorilla WebSocket v1.5.3. TLS, container networking, and external network delay were absent. The machine was not isolated from background activity, and CPU model, power state, and temperature were not controlled.

The experiment source is research/experiment/main.go. It creates an HTTP echo server and a real relay/client session. Direct mode posts to the echo server; tunnel mode posts through the relay's path route to the same server. Each response is checked for HTTP 200 and exact byte equality, including non-UTF-8 bytes. The deterministic body repeats the byte sequence 00, 01, 7f, 80, fe, ff, 22, 0a. This makes content corruption observable rather than relying solely on success status codes.

B. Workload and Measurement Procedure

The experiment crosses two payload sizes (1 KiB and 64 KiB), three concurrency levels (1, 8, and 32), two transport modes, and three repetitions. Each measured run contains 256 requests after 32 sequential warm-up requests using the same HTTP client. There are 36 measured runs, 9,216 measured requests, and 1,152 excluded warm-up requests. The 32-request warm-up initializes connections but does not fully pre-establish every concurrent connection; first-use effects can therefore remain at higher concurrency.

Workers draw jobs from a shared queue and issue the next request only after the previous one finishes. This is a closed-loop workload, not a fixed-rate open-loop load test. Reusable transports allow up to 64 idle and total connections per host. Mode order is direct-then-tunnel in repetitions one and three, and tunnel-then-direct in repetition two. This reduces a fixed-order bias but is not randomized or perfectly balanced. Each completed request is timed from immediately before HTTP POST through response-body reading and byte validation.

For a run with N requests and wall time T, throughput is N/T. Reported p50 and p95 are nearest-rank sample quantiles within each run. The tables then report the median of each metric across three repetitions. Consequently, the displayed p95 is a median of three run-level p95 values, not a pooled percentile across all samples. Per-request timings are retained in latencies.csv; run summaries and runtime metadata are retained in results.json. No confidence intervals are inferred from three repetitions.

C. Functional and Failure Experiments

Integration tests start real HTTP and WebSocket servers and exercise the implemented interfaces. The basic round-trip test checks method, path, query, a custom header, body, response header, and status code. A concurrency test submits 20 requests with staggered application delays and verifies that each response returns the correct request-specific identifier. Other tests check binary echo, redirect preservation, repeated cookies, filtering, body limits, unavailable upstreams, authentication rejection, reconnection, and cancellation. Table III groups the observed outcomes.

TABLE III OBSERVED FUNCTIONAL AND FAILURE OUTCOMES

Scenario	Observed result
Method, path, query, headers, body	Preserved in round trip
20 concurrent requests	Correct response matching
Binary, redirect, repeated cookies	Exact bytes; 302; two cookies
Connection-nominated headers	Removed in both directions
Oversize request / response	413 / 502
Stopped local application	502
Incorrect shared token	Rejected without retries
150 ms relay timeout	504; local context cancelled
Dropped WebSocket connection	Second registration observed
Wrong response owner (unit test)	Rejected; pending entry retained

The timeout test uses a 150 ms relay timeout to keep the test short; it checks for 504 and observes cancellation of the local handler's context. It tests the cancellation mechanism, not the exact wall-clock accuracy of the default 30-second policy. The reconnection test observes a second registration after an intentionally closed socket. The response-ownership unit test submits a completion under the wrong tunnel ID and confirms that the original pending request remains available to its owner.

D. Verification Scope

The ordinary suite, static checks, and race-enabled suite passed in local verification. Go's race detector only reports races that occur along executed paths [9]; a clean run is not a proof of race freedom. The tests are also not exhaustive protocol conformance, adversarial-load, browser-compatibility, or penetration tests. The present results address RQ1 and RQ2 for the tested scenarios and constrain RQ3 to a single-machine comparison.

VI. RESULTS AND ANALYSIS

A. Correctness Outcomes

All 9,216 measured requests completed with HTTP 200 and matched the submitted payload exactly. There were zero measured request failures or byte mismatches in either direct or tunnel mode. The integration checks in Table III also passed. These results support RQ1 for the sampled HTTP properties, body sizes, and code paths. They do not establish interoperability with arbitrary applications or prove that all possible malformed requests are safely handled.

The ownership, concurrent-response, timeout-cancellation, and reconnect checks support RQ2 for the tested transitions. In particular, cancelling an expired relay request was observed to cancel the corresponding local handler. This links cleanup to a directly observed effect rather than merely checking a returned error code. No quantitative recovery-time guarantee is reported, and behavior at more than 64 active client requests was not measured in the performance experiment.

B. Latency and Throughput

Table IV reports medians of the three run-level summaries. At concurrency one, tunnelled 1 KiB requests had a median latency of 0.182 ms, compared with 0.035 ms for direct HTTP. The corresponding throughput was 5,279.3 requests/s rather than 27,855.4 requests/s. For 64 KiB, the tunnel median was 3.016 ms versus 0.076 ms direct, with throughput of 331.6 versus 9,698.4 requests/s. The difference includes the entire extra path, not serialization in isolation.

TABLE IV LOCAL PERFORMANCE: MEDIANS ACROSS THREE RUNS
D = DIRECT; T = TUNNEL; LATENCY IN ms; RATE IN requests/s

KiB	C	Mode	p50	p95	Rate
1	1	D	0.035	0.046	27,855.4
1	1	T	0.182	0.284	5,279.3
1	8	D	0.090	0.318	67,579.6
1	8	T	1.036	1.444	7,807.4
1	32	D	0.492	1.808	49,641.7
1	32	T	3.197	4.989	9,595.8
64	1	D	0.076	0.204	9,698.4
64	1	T	3.016	3.138	331.6
64	8	D	0.736	1.228	10,395.0
64	8	T	11.106	11.818	717.4
64	32	D	2.131	4.839	13,213.6
64	32	T	36.943	43.961	839.2

At concurrency 32, the 1 KiB tunnel throughput reached 9,595.8 requests/s, while median latency increased to 3.197 ms and p95 to 4.989 ms. The 64 KiB tunnel reached 839.2 requests/s with median latency of 36.943 ms and p95 of 43.961 ms. Higher concurrency improves utilization in this workload, but also increases queueing and contention around shared resources. The results do not imply that increasing concurrency indefinitely improves performance.

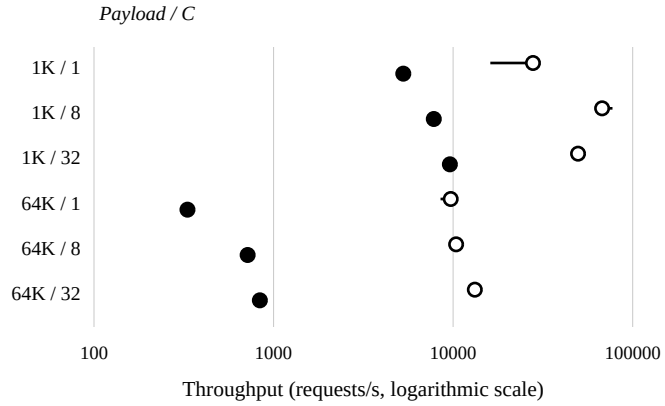

Fig. 3. Median run throughput: open circles indicate direct HTTP and filled circles indicate the tunnel. Horizontal lines show observed min-max ranges across three runs, not confidence intervals.

Figure 3 includes the minimum and maximum run throughput as descriptive ranges. For the 1 KiB direct baseline at concurrency one, throughput ranged from 16,117.1 to 27,904.8 requests/s; the tunnel ranged from 5,068.5 to 5,293.5 requests/s. This baseline variation illustrates why a single measurement would be insufficient. The plotted ranges summarize observed repetitions only; they are not confidence intervals and should not be read as population-level uncertainty estimates.

C. Interpretation of RQ3

The tunnel is consistently slower than direct HTTP in the collected conditions. This is expected because it adds HTTP handling at the relay, JSON/Base64 serialization in both directions, a shared WebSocket path, and another local HTTP exchange. Larger payloads increase representation work and transient buffering. The experiment did not profile CPU instructions, allocations, or individual pipeline stages, so it cannot establish which mechanism dominates. Binary framing and streaming are plausible optimization directions that require separate controlled experiments.

VII. DISCUSSION AND THREATS TO VALIDITY

A. What the Results Establish

The implemented design provides a compact example of reconstructing HTTP exchanges over a persistent outbound connection. The functional experiments establish the tested behavior of data representation, request correlation, and selected failure paths. The measured local overhead makes the cost of the JSON design visible. For a teaching demonstration, this inspectability is useful: students can follow a request from Go structures to JSON, through the WebSocket, and back into an HTTP request. That benefit is a design rationale, not a measured learning outcome.

The design also exposes a useful distinction between connection liveness and request completion. A healthy socket does not imply that an application request will finish, so the relay retains independent request deadlines. Similarly, authentication of a registered client does not establish ownership of every request ID; the pending map stores that ownership explicitly. These distinctions explain why heartbeat, timeout, and response-correlation mechanisms are separate parts of the implementation.

B. Internal and External Validity

Internal validity is limited by short runs, mode-order imbalance, a warm-up that does not fully populate concurrent connections, and uncontrolled background load. The workload generator shares CPU and memory with all measured services, so generator overhead can affect throughput. Checks include body reading and comparison, meaning latency is an application-observed round trip rather than a wire-only transport metric. Capturing these choices in the artifact helps reproduction, but does not remove their effects.

External validity is limited more strongly. One host, one runtime, two payload sizes, one echo operation, and three concurrency levels cannot represent typical Internet traffic. There is no cross-machine experiment, mobile network, loss injection, WAN latency, NAT traversal trial, TLS handshake measurement, or long-duration soak test. The measurements must not be presented as public deployment performance, maximum service capacity, or a comparison with ngrok or frp. Three repetitions support descriptive medians and ranges but do not establish statistical significance.

C. Implementation and Deployment Limitations

The relay stores all active state in memory and supports one process. Restarting it invalidates existing routing names; reconnection produces new URLs. Stateless load balancing across several relay replicas would not work without connection-aware routing or inter-relay forwarding. The 1,000 pending-request limit is not a per-user quota, and the shared token does not provide independent user identities, token revocation, or tenant accounting.

Path routing may require application adjustments for root-relative links, absolute redirects, cookie paths, and virtual-host assumptions. The implementation buffers complete bodies, does not rewrite arbitrary application content, and does not support server-sent events, tunnelled WebSocket upgrades, raw TCP, or gRPC streaming. HTTPS/WSS deployment instructions and a Dockerfile exist, but a public relay was not deployed for this study. The container image build was blocked by a registry certificate mismatch in the local network; native execution is the evaluated artifact.

D. Future Work

Future work should first deploy the relay on a reachable TLS endpoint and repeat measurements from a separate client network. Profiling and component ablations should then compare JSON/Base64 with binary frames while holding workload and connection management constant. Streaming with cancellation, bounded backpressure, and fair admission would extend the design to larger bodies. Per-user credentials, access policies for public callers, request-rate controls, and connection-aware multi-relay routing are prerequisites for a broader hosted service.

VIII. CONCLUSION

TunnelForge implements a reverse HTTP tunnel in Go using a versioned JSON protocol over WebSocket. Its design integrates temporary registration, ownership-aware request matching, bounded concurrent execution, cancellation, and connection recovery. Functional tests and 9,216 checked loopback requests support correctness for the evaluated scenarios. Measured overhead is substantial relative to direct HTTP, particularly for larger bodies, and motivates more efficient framing and buffering. The artifact is suitable for reproducible local systems demonstrations; Internet deployment, broad protocol compatibility, and production capacity remain unverified.

REFERENCES

[1] I. Fette and A. Melnikov, “The WebSocket Protocol,” IETF, RFC 6455, Dec. 2011, doi: 10.17487/RFC6455. Available: https://www.rfc-editor.org/rfc/rfc6455

[2] R. Fielding, M. Nottingham, and J. Reschke, Eds., “HTTP Semantics,” IETF, RFC 9110, Jun. 2022, doi: 10.17487/RFC9110. Available: https://httpwg.org/specs/rfc9110.html

[3] ngrok, “What is ngrok?” Documentation. Accessed: Oct. 29, 2026. Available: https://ngrok.com/docs/about

[4] fatedier and contributors, “frp: A fast reverse proxy,” GitHub repository. Accessed: Oct. 29, 2026. Available: https://github.com/fatedier/frp

[5] T. Bray, Ed., “The JavaScript Object Notation (JSON) Data Interchange Format,” IETF, RFC 8259, Dec. 2017, doi: 10.17487/RFC8259. Available: https://www.rfc-editor.org/rfc/rfc8259

[6] S. Josefsson, “The Base16, Base32, and Base64 Data Encodings,” IETF, RFC 4648, Oct. 2006, doi: 10.17487/RFC4648. Available: https://www.rfc-editor.org/rfc/rfc4648

[7] The Go Authors, “Package encoding/json,” Go package documentation. Accessed: Oct. 29, 2026. Available: https://pkg.go.dev/encoding/json

[8] Gorilla WebSocket contributors, “Package websocket,” Go package documentation. Accessed: Oct. 29, 2026. Available: https://pkg.go.dev/github.com/gorilla/websocket

[9] The Go Authors, “Data Race Detector,” Go documentation. Accessed: Oct. 29, 2026. Available: https://go.dev/doc/articles/race_detector